

Caderno de Exercícios: Classes Abstratas e Limitações de Herança

Dani Juliano Czelusniak, Prof. Dr.

Atualizado em 2026

1 Apresentação

Este caderno de laboratório contém 6 exercícios práticos inéditos, elaborados para explorar os conceitos de **Classes Abstratas**, **Interfaces** e as **Limitações de Herança em Java**.

Você praticará desde o gerenciamento de estado em classes abstratas até a contínua adoção de múltiplos contratos via interfaces. Além disso, os níveis avançados cobrem a resolução manual de conflitos arquiteturais como o *Problema do Diamante*.

2 Nível Básico

Exercício 1 (Herança Simples com Estado)

Como vimos, classes abstratas resolvem o "Problema do Estado" pois permitem que apenas UMA classe as estenda. Crie a classe abstrata `EquipamentoEletronico` que possua um estado protegido `voltagem` e um método abstrato `ligar()`. Crie a classe `AspiradorDePo` que estende o equipamento e implementa a inicialização. Teste-os no `main`.

Exercício 2 (Interfaces como Contratos Simples)

Interfaces contornam o problema da ambiguidade por ditarem apenas o "que" deve ser feito. Crie a interface `Conectavel` com o método `conectarInternet()`. Crie a classe `SmartTV` que implementa essa interface simulando uma conexão via Wi-Fi.

3 Nível Intermediário

Exercício 3 (Herança Múltipla de Comportamento)

Java não permite herdar de duas classes, mas permite implementar várias interfaces! Crie as interfaces `Camera` (com o método `tirarFoto()`) e `GPS` (com o método `obterLocalizacao()`). Crie a classe `Smartphone` que adota os dois contratos simultaneamente.

Exercício 4 (Combinando Extends e Implements)

Um design comum é herdar características gerais e plugar habilidades específicas. Crie a classe abstrata `Funcionario` com o método abstrato `trabalhar()`. Crie a interface

`AcessoAdministrativo` com o método `entrarNoServidor()`. Crie a classe `GerenteRede` que estende o funcionário e assina a interface simultaneamente.

4 Nível Avançado

Exercício 5 (A Volta do Problema do Diamante no Java 8+)

Com a chegada dos métodos `default`, as interfaces podem ter corpo, trazendo de volta o risco da ambiguidade. Crie duas interfaces: `Drone` e `CameraAerea`, ambas contendo o MESMO método `default void calibrar()`. Crie a classe `EquipamentoFilmagens` que implementa ambas. O compilador exigirá que você reescreva o método para resolver o conflito manualmente usando o `super` específico.

Exercício 6 (Ecossistema de Interfaces e Downcasting)

Crie a interface `Armazenamento` com o método `salvar(String dados)`. Crie as classes `DiscoRigido` e `SSD` que implementam a interface (o `SSD` deve adicionar um método próprio `otimizar()`). No `main`, crie um vetor de `Armazenamento` com ambos. Itere sobre eles salvando os dados e, usando `instanceof`, descubra qual deles é o `SSD` para acionar o seu método específico de otimização.